

InferaCordon — Revised Detailed Architecture Plan

Preamble: Why This Revision Exists

Every prior version of this document accumulated components faster than they could be validated. The result was a proposal that was technically impressive on paper but increasingly disconnected from what one engineer can build, test, and demonstrate in 14 weeks on a single GPU node.

This revision applies one governing principle to every decision: **if a component cannot be built, tested, and demonstrated to work correctly within its allocated phase, it does not belong in the MVP.**

The problem statement is preserved without compromise. The architecture is fixed — meaning finalized, not repaired — with precise component boundaries, explicit interface contracts, and an implementation sequence that front-loads risk so failures are discovered in Week 1, not Week 13.

The Problem Statement (Canonical, Final)

Reasoning models generate internal chain-of-thought tokens before producing any visible output. These tokens are billed in full, consume context window, dominate inference latency, and are completely invisible to every standard monitoring, cost attribution, and governance tool in the LLMOps ecosystem.

Platform teams have no control surface for this new spend. They set one global `max_tokens` ceiling for all requests, all tenants, and all traffic conditions. The result is three simultaneous failures:

Overthinking wastes compute. A model spends 1,800 thinking tokens on a query requiring 200. No mechanism exists to recognize this and stop earlier.

Underthinking degrades quality silently. A complex query hits a conservative global ceiling. The model produces a shallow answer. No error is raised. No alert fires.

Budget blindness makes both failures invisible. There is no per-request allocation, no tenant-level cost attribution, no SLO-aware routing, and no audit trail recording what was spent and why.

InferaCordon builds the control plane that treats reasoning tokens as a managed compute resource — allocatable, policy-governed, cost-attributed, quality-verified, and reliably observed.

Governing Constraints

These constraints are not preferences. They are hard boundaries that govern every architectural decision.

One engineer. Fourteen weeks. One GPU node. Nothing in the architecture requires more than this.

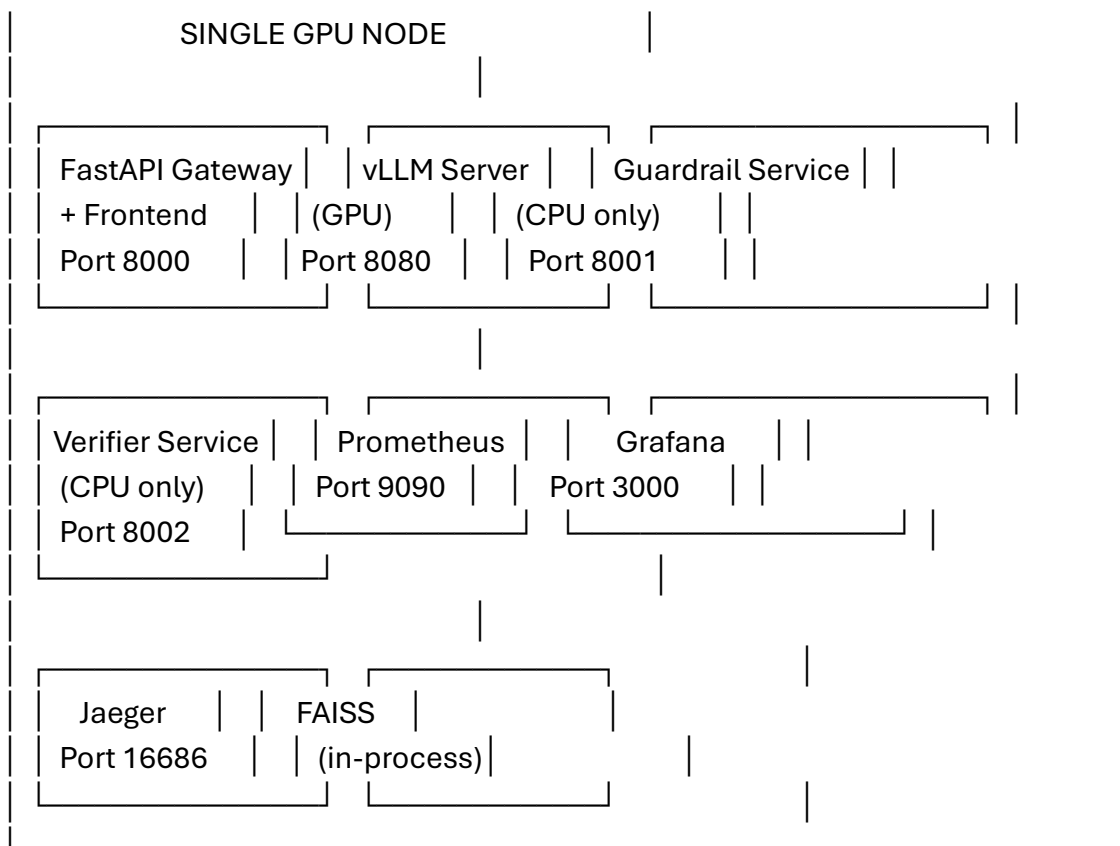
One reasoning model. One cheap model. DeepSeek-R1-7B-Q4 for reasoning. Qwen2.5-3B-Instruct for cheap routing. Both on the same vLLM instance.

No component enters production without a passing validation test. The LogitProcessor is validated in Week 1. The guardrail service is validated before it touches production traffic. Every failure mode is tested before the demo is recorded.

No claim appears in the README that is not produced by a benchmark script. Every number is measured. None is estimated or projected.

All four pillars must be demonstrable in a five-minute live demo. Guardrails, Governance, Cost Optimization, and Inference Optimization each need at least one concrete, visible, interactive demonstration in the frontend.

System Topology



Six containers. All started by docker compose up. The gateway container hosts the FastAPI application, the React frontend static files, the in-process FAISS semantic cache, and the background evaluation thread pool. No separate Redis. No separate Celery. No separate frontend server.

Component 1: FastAPI Gateway

Responsibility Boundary

The gateway owns the complete request lifecycle from client receipt to client response. It is the only component that talks directly to the client. Every other component is called by the gateway. The gateway never delegates control flow to another component — it calls services, receives results, and makes decisions.

Internal Module Structure

gateway/

- ├─ main.py ← FastAPI app, lifespan, route registration
- ├─ middleware/
- │ └─ auth.py ← API key → tenant ID resolution
- │ └─ rate_limiter.py ← Token bucket, in-memory, per-tenant
- │ └─ trace_injector.py ← UUID request ID, W3C traceparent
- ├─ pipeline/
- │ └─ orchestrator.py ← Pre-inference pipeline, 50ms deadline
- │ └─ complexity_scorer.py ← Heuristic scorer, <3ms
- │ └─ pii_redactor.py ← Presidio wrapper
- │ └─ context_engine.py ← Static prompt template selection
- ├─ routing/
- │ └─ budget_controller.py ← Budget class assignment
- │ └─ model_router.py ← Cheap vs reasoning route selection

- | |— admission_control.py ← Semaphore, queue depth
- | |— semantic_cache.py ← FAISS index, sentence-transformers
- | |— clients/
- | |— vllm_client.py ← OpenAI-compatible HTTP client
- | |— guardrail_client.py ← HTTP client for guardrail service
- | |— verifier_client.py ← HTTP client for verifier service
- | |— escalation/
- | |— escalation_handler.py ← Retry at higher budget class
- | |— evaluation/
- | |— eval_worker.py ← Background thread pool, SQLite writes
- | |— static/ ← Built React frontend

Request Processing Sequence

Step 1 — Authentication (sync, <1ms)

The auth middleware resolves the Authorization header API key against an in-memory dictionary loaded from tenants.yaml at startup. Unknown keys return 401 immediately. The resolved tenant ID is attached to the request state object and propagates to every subsequent step.

Step 2 — Rate Limiting (sync, <1ms)

The rate limiter checks two token buckets for the tenant: requests per minute and tokens per hour. Both buckets refill continuously using the token bucket algorithm. If either bucket is empty, the gateway returns 429 with a structured body:

```
{
  "error": "rate_limit_exceeded",
  "limit_type": "requests_per_minute",
  "retry_after_seconds": 14,
  "tenant_id": "acme_corp",
  "policy_version": "enterprise_standard_v1"
}
```

Step 3 — Trace Context Injection (sync, <1ms)

A UUID request ID is generated. A W3C traceparent header is constructed from this ID and attached to the request state. All downstream log statements, span creations, and audit records reference this ID. A request that fails at any point produces a complete audit trail back to this origin.

Step 4 — Pre-Inference Pipeline (sync, 50ms deadline)

The pipeline orchestrator starts a deadline timer and runs three operations. If the cumulative time reaches 48ms before all three complete, the orchestrator cancels the remaining operations and sets the fast-path flag. The request proceeds with default medium budget class and complexity score -1.

The three operations run sequentially because each feeds the next:

Complexity scoring runs first and produces a score that policy lookup uses for SLO class selection. PII redaction runs last because it modifies the prompt text that all subsequent components see.

Step 5 — Policy Lookup (sync, <1ms)

The policy engine returns the versioned policy document for the tenant and domain. This is an in-memory dictionary lookup. The policy document contains all parameters needed for the remaining steps: budget profiles, guardrail configuration, rate limits, cost ceilings, latency SLOs, escalation settings, and version identifiers.

Step 6 — Budget Class Assignment (sync, <1ms)

The budget controller maps the complexity score to a budget class using the policy's budget profiles. It also assigns a priority tier by evaluating three independent dimensions: complexity (estimated reasoning demand), risk (configured in policy per domain), and scheduling pressure (current fleet queue depth). Priority tier governs scheduling preference under load, not answer quality budget.

Step 7 — Decision Record Write (async, non-blocking)

The policy engine writes the pre-inference decision record to the audit log. This write is non-blocking — the request does not wait for the write to complete. The record is queued to a background write thread. If the write queue is full (bounded at 1,000 entries), the write is dropped with a counter increment. Lost audit records are logged as a warning metric but do not block serving.

Step 8 — Semantic Cache Lookup (sync, <5ms)

The FAISS index is queried with the redacted prompt embedding. If cosine similarity exceeds the policy threshold, the cached response is returned immediately. The cache hit is recorded in the outcome record. No guardrail, no inference, no verification.

Step 9 — Admission Control (sync, <1ms)

The admission controller checks the current count of active reasoning model requests against the configured semaphore ceiling. For low-priority requests when the ceiling is reached, the request is queued with a configurable timeout. For high-priority requests, the timeout is extended. If the queue timeout is exceeded, the gateway returns 503.

Step 10 — Guardrail Input Check (sync, network call)

The gateway calls the guardrail service /input endpoint with the redacted prompt and the tenant's guardrail configuration. The call has a 200ms timeout. If the call times out, the gateway applies a conservative block and returns 503. If the call succeeds, the result is attached to the request state.

Step 11 — Inference (sync, network call)

The gateway calls the vLLM server's OpenAI-compatible endpoint with the request parameters including the budget class token ceiling and the registered LogitProcessor identifier. The call streams the response. The gateway buffers the stream and detects the stop reason from the completion metadata.

Step 12 — Guardrail Output Check (sync or async based on policy)

For high-risk policy domains, the output guardrail check is synchronous — the response is not delivered until the safety check completes. For standard domains, the output check runs asynchronously after response delivery and the result is written to the outcome record.

Step 13 — Verification (sync, network call)

If the policy requires verification for the assigned budget class, the gateway calls the verifier service with the response and the expected format or ground truth. A failed verification triggers escalation.

Step 14 — Escalation (sync, conditional)

If verification fails and the current budget class is not already at maximum, the gateway increments the escalation counter and repeats steps 9-13 at the next higher budget class. Maximum one retry in the MVP. All retry costs are attributed to the original request. If the retry also fails verification, the gateway returns the response with a confidence_level: low field added to the response body.

Step 15 — Response Delivery

The response is delivered to the client. Immediately after delivery, the outcome record is queued to the audit log write thread and the evaluation task is queued to the background evaluation worker.

Step 16 — Async Evaluation (background, post-response)

The evaluation worker thread picks up the task, calls the LLM-as-judge API for non-verifiable responses, writes the result to SQLite, and updates the Prometheus quality metrics. This never blocks the response path.

Interface Contract: Gateway API

POST /v1/infer

Headers:

Authorization: Bearer <api_key>

Content-Type: application/json

Body:

```
{
  "prompt": string,
  "tenant_id": string,    // redundant with API key; for audit
  "domain": string,      // maps to policy selection
  "response_schema": object, // optional; triggers JSON verification
  "budget_override": string, // optional; "low"|"med"|"high"|"crit"
  "stream": boolean      // optional; default false
}
```

Response:

```
{
  "response": string,
  "request_id": string,
  "budget_class": string,
  "reasoning_tokens_used": integer,
  "reasoning_tokens_allocated": integer,
  "stop_reason": string,
  "verification_result": string,
  "escalation_count": integer,
  "estimated_cost_usd": float,
  "confidence_level": string, // "high"|"low"
  "latency_ms": integer
}
```

GET /v1/health

GET /metrics // Prometheus scrape endpoint

GET /audit/records // Audit log query endpoint

GET /admin/policies // Policy inspection endpoint

POST /admin/inject-failure // Demo mode only

Component 2: Policy Engine

Responsibility Boundary

The policy engine is a Python library, not a service. It is imported directly by the gateway and runs in-process. Its responsibilities are: loading and validating policies at startup, resolving the correct policy for a tenant and domain at request time, computing budget allocations, and writing audit records.

Policy Schema (Complete)

Policy file: policies/acme_corp_general_qa_v1.yaml

policy_id: enterprise_standard_v1
tenant: acme_corp
domain: general_qa
schema_version: 1

— Budget Profiles

budget_profiles:

low:

model: qwen25-3b
max_reasoning_tokens: 0 # No thinking; cheap model
max_output_tokens: 256
verification: none
context_template: direct

medium:

model: deepseek-r1-7b
max_reasoning_tokens: 512
max_output_tokens: 512
verification: verifiable_only
context_template: step_by_step

high:

model: deepseek-r1-7b
max_reasoning_tokens: 1024
max_output_tokens: 1024
verification: required
context_template: verify_steps

critical:

model: deepseek-r1-7b
max_reasoning_tokens: 2048
max_output_tokens: 1024
verification: required
context_template: verify_steps

— Complexity Thresholds

complexity_thresholds:

low_max: 3.5
medium_max: 6.5
high_max: 8.5
> 8.5 → critical

— Guardrails

guardrails:

pii_redaction: standard # always; presidio
injection_check: conditional # only on tool_use domain
safety_check: conditional # only on policy-flagged domains
output_safety: async # "sync" for high-risk domains

— Rate Limits —————

limits:

requests_per_minute: 60
tokens_per_hour: 500000
max_cost_per_request_usd: 0.05
max_prompt_tokens: 4096

— SLOs —————

slo:

p95_latency_ms: 4000
quality_floor_score: 3.5 # LLM judge score floor (1-5)
quality_regression_pp: 1.0 # Max accuracy drop vs. baseline

— Cache —————

cache:

enabled: true
similarity_threshold: 0.92
ttl_seconds: 3600

— Escalation —————

escalation:

enabled: true
max_retries: 1
on_exhaustion: return_low_confidence

— Versions —————

Every component version that governs this policy is recorded.

A quality regression is traceable to an exact version change.

versions:

policy: 1
system_prompt: prompt_v2
complexity_scorer: scorer_v1
guardrail_model: llamaguard_onnx_v1
injection_model: deberta_inject_v1
verifier: verifier_v1

— Data Handling —————

trace_retention_days: 30
store_reasoning_trace: false # Never store raw chain-of-thought

Pydantic Validation

Every field has a type annotation, a validator, and a default where appropriate. The policy loader calls `PolicyDocument.model_validate(yaml_data)` for each file at startup. Validation failure raises a startup exception with a field-level error message. The gateway does not start with an invalid policy file.

Budget Controller Logic

```
def assign_budget_class(
    complexity_score: float,
    policy: PolicyDocument,
    fleet_state: FleetState
) -> BudgetDecision:

    # Map complexity score to base class using policy thresholds
    if complexity_score <= policy.complexity_thresholds.low_max:
        base_class = "low"
    elif complexity_score <= policy.complexity_thresholds.medium_max:
        base_class = "medium"
    elif complexity_score <= policy.complexity_thresholds.high_max:
        base_class = "high"
    else:
        base_class = "critical"

    # Apply fleet pressure downgrade for non-critical requests
    # Priority is scheduling preference; it does not reduce quality budget
    if fleet_state.latency_cb_open and base_class != "critical":
        effective_class = downgrade_one(base_class)
        downgrade_reason = "latency_circuit_breaker"
    elif fleet_state.gpu_pressure_cb_open and base_class == "low":
        effective_class = "low" # already cheap route; no change
        downgrade_reason = None
    else:
        effective_class = base_class
        downgrade_reason = None

    profile = policy.budget_profiles[effective_class]

    return BudgetDecision(
        base_class=base_class,
        effective_class=effective_class,
        downgrade_reason=downgrade_reason,
        model=profile.model,
        max_reasoning_tokens=profile.max_reasoning_tokens,
        max_output_tokens=profile.max_output_tokens,
        verification=profile.verification,
        context_template=profile.context_template,
    )
```

Audit Log Record Schema

Two records per request. Written to separate append-only JSONL files.

Decision record (written before inference):

```
{
  "record_type": "decision",
  "timestamp_iso": "2026-08-09T10:42:11.334Z",
  "request_id": "req_8c4f2a",
  "tenant_id": "acme_corp",
  "domain": "general_qa",
  "policy_id": "enterprise_standard_v1",
  "policy_version": 1,
  "model_version": "deepseek-r1-7b-q4",
  "prompt_version": "prompt_v2",
  "guardrail_version": "llamaguard_onnx_v1",
  "verifier_version": "verifier_v1",
  "complexity_score": 6.4,
  "fast_path_used": false,
  "base_budget_class": "medium",
  "effective_budget_class": "medium",
  "downgrade_reason": null,
  "priority_tier": "standard",
  "max_reasoning_tokens": 512,
  "max_output_tokens": 512,
  "route": "reasoning_model",
  "cache_hit": false,
  "admission_result": "pass",
  "estimated_cost_usd": 0.0062
}
```

Outcome record (written after response delivery):

```
{
  "record_type": "outcome",
  "timestamp_iso": "2026-08-09T10:42:12.501Z",
  "request_id": "req_8c4f2a",
  "reasoning_tokens_used": 341,
  "reasoning_tokens_allocated": 512,
  "tokens_saved": 171,
  "output_tokens": 203,
  "stop_reason": "natural_boundary",
  "escalation_count": 0,
  "fallback_used": false,
  "guardrail_input_result": "pass",
  "guardrail_output_result": "pass",
  "verification_result": "pass",
  "actual_cost_usd": 0.0058,
  "queue_ms": 18,
  "guardrail_input_ms": 24,
```

```
"prefill_ms": 187,  
"decode_ms": 810,  
"ttft_ms": 205,  
"e2e_latency_ms": 1167,  
"circuit_breaker_state_at_completion": "closed"  
}
```

Component 3: Complexity Scorer

Responsibility Boundary

Produces a float complexity score from 0.0 to 10.0 from a text prompt in under 3ms on CPU. No ML model inference on the critical path. No network call. No external dependency.

Implementation

The scorer extracts seven feature groups from the prompt text and combines them using a weighted sum calibrated against the baseline benchmark data collected in Week 1.

Feature Group 1: Constraint Keyword Density

A vocabulary of constraint-indicating terms partitioned into three tiers by cognitive weight.

Tier 3 (weight 1.5 each): prove, derive, demonstrate that, show that, verify that, find all, for all, if and only if, necessary and sufficient.

Tier 2 (weight 1.0 each): compare, contrast, analyze, evaluate, explain why, step by step, what are the implications, both, unless, however, except when.

Tier 1 (weight 0.5 each): describe, summarize, list, what is, who, when, where.

Score contribution: sum of (term count × tier weight), normalized to 0-3.

Feature Group 2: Structural Signals

Question mark count (normalized: 0 questions = 0, 3+ questions = 1.0). Sentence count (0 sentences = 0, 10+ sentences = 1.0). Bullet point or numbered list item count (normalized). Explicit multi-step instruction pattern (regex: "first ... then ... finally" or "step 1", "step 2" etc.) as binary 0 or 1.

Score contribution: weighted sum, normalized to 0-2.

Feature Group 3: Technical Domain Signals

Code block presence (markdown triple backtick or four-space indent): binary 1.0. Mathematical notation presence (LaTeX dollar signs, Greek letters, equation patterns): binary 1.0. Formal logic notation ($\forall$, $\exists$, $\rightarrow$, $\leftrightarrow$): binary 1.5. JSON or XML structure present in prompt: binary 0.5.

Score contribution: sum, capped at 2.0.

Feature Group 4: Length Signal

Prompt token count estimated by character count divided by 4 (fast approximation). Short prompts (<50 tokens) contribute 0. Medium (50-200 tokens) contribute 0.5. Long (200-500 tokens) contribute 1.0. Very long (>500 tokens) contribute 1.5.

Score contribution: 0-1.5.

Feature Group 5: Output Format Complexity

Request for structured output (JSON, table, numbered list, code): 0.5. Request for multiple sections or headers: 0.5. Request for a specific format with constraints: 1.0.

Score contribution: 0-1.5.

Final Score Computation

```
raw_score = (  
    constraint_score    # 0-3.0  
    + structural_score  # 0-2.0
```

```
+ domain_score    # 0-2.0
+ length_score    # 0-1.5
+ format_score    # 0-1.5
)
# raw_score range: 0-10
```

```
final_score = min(10.0, max(0.0, raw_score))
```

Calibration

During Week 1, natural completion token counts are collected for all benchmark queries. The feature weights are adjusted using a simple linear regression to minimize the mean absolute error between the scorer's budget class prediction and the budget class that would have been optimal given the observed token counts. This calibration is done once, offline, and the resulting weights are committed to the codebase.

Component 4: Semantic Cache

Responsibility Boundary

Avoids inference entirely for requests with high semantic similarity to recent queries. In-process FAISS flat index. No external service.

Implementation

The sentence-transformer model (all-MiniLM-L6-v2, 22MB, CPU) is loaded at gateway startup and held in memory. For each request, the redacted prompt is embedded in approximately 15ms on CPU. The FAISS IndexFlatIP index is queried for the top-1 nearest neighbor. If the inner product similarity exceeds the policy threshold, the cached response is returned.

Cache entries are stored in a Python dictionary keyed by FAISS vector index position. Each entry contains the response text, the original request metadata, the timestamp, and the expiry timestamp computed from the policy TTL. A background thread sweeps expired entries every 60 seconds and removes them from both the FAISS index and the dictionary.

The FAISS index is persisted to a file on a Docker volume every 5 minutes. Gateway restart restores the index from the persisted file, preserving cache state across restarts.

Cache hit rate, miss rate, and TTL expiration rate are tracked as labeled Prometheus counters. The Prometheus labels include tenant ID and domain, enabling per-tenant cache effectiveness analysis on the Governance dashboard panel.

Important scoping note: The semantic cache only applies when the policy's `cache.enabled` field is true. High-risk policy domains (where content freshness or uniqueness is critical) set `cache.enabled: false`. The cache is never used for requests that contain PII in the post-redaction prompt (logged as a warning; PII in prompts after redaction indicates a Presidio miss and should be investigated).

Component 5: vLLM Inference Server and LogitProcessor

Responsibility Boundary

Serves two models. Enforces per-request reasoning token budgets natively. Exposes Prometheus metrics. Never receives raw prompts — only redacted, policy-validated prompts from the gateway.

Model Configuration

DeepSeek-R1-7B-Q4 is loaded as the primary reasoning model. Qwen2.5-3B-Instruct is loaded as the cheap model. Both are served via a single vLLM instance using its multi-model routing capability. The gateway selects the model by passing the model name in the API call's model field.

Both models are quantized to 4-bit using GPTQ or AWQ quantization. This is required to fit both models on a single A100 80GB GPU with sufficient KV-cache headroom for concurrent requests. The quantization format is tested during Week 1 baseline setup and the best-performing format (lowest quality regression on MATH-500) is selected.

LogitProcessor Design

The LogitProcessor is the single most important technical component in the system. It is validated before any other component is built. Its design must be fixed here without ambiguity.

Registration: The LogitProcessor is a Python class implementing vLLM's LogitsProcessor interface. It is instantiated per-request and passed to vLLM via the SamplingParams.logits_processors list. Each request gets its own LogitProcessor instance, which carries its own state. There is no shared mutable state between request instances.

State per instance:

@dataclass

class BudgetLogitProcessor:

 request_id: str

 max_reasoning_tokens: int

 end_of_thinking_token_id: int # Model-specific; found in Week 1

 reasoning_complete: bool = False

 reasoning_token_count: int = 0

 entropy_log: list[float] = field(default_factory=list)

Per-step behavior:

def __call__(

 self,

 input_ids: torch.Tensor,

 scores: torch.Tensor

) -> torch.Tensor:

 if self.reasoning_complete:

 # Already transitioned to answer phase; no intervention

 return scores

 self.reasoning_token_count += 1

 # Log entropy for telemetry (monitoring only, not control)

 probs = torch.softmax(scores, dim=-1)

 top_k_probs = torch.topk(probs, k=20).values

 entropy = -torch.sum(

 top_k_probs * torch.log(top_k_probs + 1e-9)

).item()

 self.entropy_log.append(entropy)

 # Check if model emitted delimiter naturally

 last_token = input_ids[-1].item()

 if last_token == self.end_of_thinking_token_id:

 self.reasoning_complete = True

```
return scores
```

```
# Enforce budget ceiling
```

```
if self.reasoning_token_count >= self.max_reasoning_tokens:
```

```
    # Force the end-of-thinking delimiter token
```

```
    scores[:] = -float('inf')
```

```
    scores[self.end_of_thinking_token_id] = 100.0
```

```
    self.reasoning_complete = True
```

```
return scores
```

End-of-thinking token identification: DeepSeek-R1-7B uses a specific token for `</think>`. The exact token ID is determined during Week 1 by running the tokenizer:

```
tokenizer = AutoTokenizer.from_pretrained("deepseek-ai/DeepSeek-R1-Distill-Qwen-7B")
```

```
end_token_id = tokenizer.convert_tokens_to_ids("</think>")
```

This token ID is stored as a constant in `vllm_adapter/constants.py` and imported by the `LogitProcessor`.

Validation test (Week 1 gate): Fifty diverse queries are run with forced delimiter injection at five different token positions: 25%, 50%, 75%, 100% of budget, and natural completion. For each, the response is scored for coherence and completeness using the LLM judge. If the coherence rate at any forced position falls below 85%, the implementation falls back to hard `max_tokens` truncation. This fallback is simpler (no `LogitProcessor` required), less elegant, but completely reliable. The project does not proceed to Phase 2 with an unvalidated stopping mechanism.

Entropy logging: The entropy log accumulated per request is stored in the request's outcome metadata and written to the OpenTelemetry trace as a sampled array (every 10th value to limit trace size). This feeds the notebooks/entropy_signal_analysis.ipynb notebook that runs offline during Phase 5 to determine whether entropy-based adaptive early stopping is worth implementing as a future extension.

Circuit Breakers

Two circuit breakers are implemented as a Python class with thread-safe state management.

GPU Pressure Circuit Breaker:

State: CLOSED | OPEN

Metric: `vllm:gpu_cache_usage_perc` (scraped from vLLM Prometheus)

Open condition: `gpu_cache_usage_perc > 90.0%` for 2 consecutive scrapes

Close condition: `gpu_cache_usage_perc < 80.0%` for 3 consecutive scrapes

Scrape interval: 5 seconds

Effect when OPEN: Low-priority requests (`priority_tier == "low"`) are rerouted to the cheap model regardless of budget class.

High and standard priority requests are unaffected.

Latency Circuit Breaker:

State: CLOSED | OPEN

Metric: P99 TTFT from vLLM Prometheus histogram

Open condition: `P99 TTFT > policy slo.p95_latency_ms` for 3 consecutive evaluation windows (window = 30 seconds)

Close condition: `P99 TTFT < policy slo.p95_latency_ms * 0.85` for 2 consecutive windows

Effect when OPEN: All new requests are assigned one budget class lower than their calculated class. "low" class requests are rejected with 503 + retry guidance.

Both circuit breaker state transitions are emitted as OpenTelemetry span events and as Prometheus gauge changes. The frontend System Health page polls the gauge metrics every 10 seconds to update the circuit breaker status indicators.

Component 6: Guardrail Service

Responsibility Boundary

A separate FastAPI container running exclusively on CPU. Receives redacted prompt text and policy guardrail configuration. Returns a structured result indicating pass, flag, or block, with a reason and a confidence score. Never touches the GPU. Never calls vLLM.

Container Isolation Rationale

The guardrail service runs in a separate container for three reasons. First, guardrail models must not compete with inference models for GPU VRAM — that competition would degrade both. Second, the guardrail service has a different failure mode from the inference engine: a crashed guardrail container produces a defined conservative-block behavior in the gateway, whereas a crashed vLLM container produces an inference outage. Third, independent Prometheus metrics for the guardrail service allow it to be monitored, alarmed, and capacity-planned separately.

Model Loading

Two ONNX models are loaded at container startup and held in memory for the process lifetime.

Injection classifier: A DeBERTa-v3-small model fine-tuned for prompt injection detection, exported to ONNX with int8 quantization. Input: prompt text up to 512 tokens. Output: float risk score 0.0 to 1.0. Inference time on CPU: approximately 30-50ms per request.

Safety classifier: A Llama Guard 3-1B model variant exported to ONNX. Input: prompt or response text. Output: category label (safe, unsafe) and confidence score. Inference time on CPU: approximately 80-120ms per request.

Both models are mounted as Docker volumes. The container fails startup if the volume mount is absent or the models cannot be parsed by ONNX Runtime.

Service Interface

POST /guardrail/input

```
Body: {
  "text": string,           // redacted prompt
  "run_injection_check": boolean, // from policy
  "run_safety_check": boolean,   // from policy
  "injection_threshold": float,   // from policy
  "safety_threshold": float      // from policy
}

Response: {
  "result": "pass" | "flag" | "block",
  "injection_score": float,
  "safety_category": string,
  "safety_confidence": float,
  "latency_ms": integer
}
```

POST /guardrail/output

Body: {

"text": string, // final answer text

"run_safety_check": boolean,

"safety_threshold": float

}

Response: {

"result": "pass" | "block",

"safety_category": string,

"safety_confidence": float,

"latency_ms": integer

}

GET /health

GET /metrics

Decision Logic

Input guardrail decision:

if injection_score > injection_threshold:

result = "flag"

action = augment_system_prompt

(not block; false positive risk too high)

if safety_confidence > safety_threshold AND category != "safe":

result = "block"

action = return_structured_refusal

else:

result = "pass"

Output guardrail decision:

if safety_confidence > safety_threshold AND category != "safe":

result = "block"

action = return_structured_refusal

else:

result = "pass"

The injection classifier never blocks. It only flags. A flagged request proceeds to inference with an additional system prompt instruction reinforcing the original task boundary. This decision reflects the practical reality that prompt injection classifiers have non-trivial false positive rates on complex legitimate instructions, and blocking legitimate enterprise requests is a worse failure mode than the injection risk.

Timeout Handling

The gateway sets a 200ms HTTP timeout on all guardrail service calls. A timeout is treated as a conservative block for input checks (the request is not processed) and as a non-blocking pass for output checks (the response is delivered and the check failure is logged as an audit event). This

asymmetry reflects the risk difference: it is safer to block an unverified input than to hold a completed response waiting for a check that may never complete.

Component 7: Verifier Service

Responsibility Boundary

A separate FastAPI container running exclusively on CPU. Wraps official evaluation harnesses.

Returns a structured verification result. Never implements verification logic from scratch — always delegates to the canonical evaluation tool for each task type.

Verification Implementations

GSM8K verifier:

```
import re
```

```
def verify_gsm8k(response: str, ground_truth: str) -> VerificationResult:
```

```
    # Extract final number from response
```

```
    # Pattern matches the last number appearing after common
```

```
    # answer-indication phrases
```

```
    pattern = r"(?:answer is|=|equals|result is|total is)?\s*([-\\d,]+\\.?\\d*)\s*$"
```

```
    matches = re.findall(r"([-\\d,]+\\.?\\d*", response)
```

```
    if not matches:
```

```
        return VerificationResult(passed=False, reason="no_number_found")
```

```
    predicted = float(matches[-1].replace(",", ""))
```

```
    expected = float(ground_truth.replace(",", ""))
```

```
    passed = abs(predicted - expected) < 1e-6
```

```
    return VerificationResult(
```

```
        passed=passed,
```

```
        predicted=str(predicted),
```

```
        expected=str(expected),
```

```
        reason="exact_match" if passed else "numeric_mismatch"
```

```
)
```

MATH-500 verifier:

```
import sympy
```

```
def verify_math(response: str, ground_truth: str) -> VerificationResult:
```

```
    # Extract content within \boxed{} if present
```

```
    boxed_pattern = r"\\boxed\\{([\\^}]+)\\}"
```

```
    match = re.search(boxed_pattern, response)
```

```
    predicted_str = match.group(1) if match else response.strip().split("\\n")[-1]
```

```
    # Attempt exact string match after normalization
```

```
    norm_predicted = normalize_math_str(predicted_str)
```

```
    norm_expected = normalize_math_str(ground_truth)
```



```

if norm_predicted == norm_expected:
    return VerificationResult(passed=True, reason="exact_match")

# Attempt SymPy symbolic equivalence
try:
    expr_predicted = sympy.sympify(norm_predicted)
    expr_expected = sympy.sympify(norm_expected)
    passed = sympy.simplify(expr_predicted - expr_expected) == 0
    return VerificationResult(
        passed=passed,
        reason="symbolic_equiv" if passed else "symbolic_mismatch"
    )
except Exception:
    return VerificationResult(passed=False, reason="parse_failed")

```

HumanEval verifier:

The HumanEval test runner executes in a sandboxed subprocess with strict resource limits. The sandbox is a subprocess with resource.setrlimit applied before execution: CPU time limit 2 seconds, memory limit 256MB, no network access via os.setuid to a restricted user, filesystem writes restricted to /tmp.

```

import subprocess
import tempfile
import os

```

```

def verify_humaneval(
    response: str,
    test_cases: str,
    entry_point: str
) -> VerificationResult:

    code = f'{{response}}\n\n{{test_cases}}\n\ncheck({{entry_point}})'

    with tempfile.NamedTemporaryFile(
        mode='w', suffix='.py', delete=False
    ) as f:
        f.write(code)
        tmpfile = f.name

    try:
        result = subprocess.run(
            ["python", "-u", tmpfile],
            capture_output=True,
            timeout=2.0,
            text=True
        )
        passed = result.returncode == 0

```

```

    return VerificationResult(
        passed=passed,
        reason="tests_passed" if passed else "tests_failed",
        stderr=result.stderr[:500] if not passed else None
    )
except subprocess.TimeoutExpired:
    return VerificationResult(passed=False, reason="timeout")
finally:
    os.unlink(tmpfile)
JSON schema verifier:
import jsonschema

def verify_json_schema(
    response: str,
    schema: dict
) -> VerificationResult:
    try:
        data = json.loads(response)
    except json.JSONDecodeError as e:
        return VerificationResult(passed=False, reason=f"invalid_json: {e}")

    try:
        jsonschema.validate(data, schema)
        return VerificationResult(passed=True, reason="schema_valid")
    except jsonschema.ValidationError as e:
        return VerificationResult(
            passed=False,
            reason=f"schema_violation: {e.message}"
        )

```

Service Interface

POST /verify

Body: {

```

    "response": string,
    "verifier_type": "gsm8k"|"math"|"humaneval"|"json_schema"|"none",
    "ground_truth": string,      // for gsm8k, math
    "test_cases": string,       // for humaneval
    "entry_point": string,      // for humaneval
    "schema": object            // for json_schema
}

```

Response: {

```

    "passed": boolean,
    "verifier_type": string,
    "reason": string,
    "predicted": string,
    "expected": string,

```

```
"latency_ms": integer
}
```

GET /health

GET /metrics

Component 8: Observability Stack

OpenTelemetry Trace Schema

Every request produces one trace with spans for each processing stage. The complete schema is defined in `telemetry/trace_schema.py` as a dataclass and imported by all components. This guarantees consistency — no component invents its own field names.

```
@dataclass
```

```
class RequestTrace:
```

```
    # Identity
```

```
    request_id: str
```

```
    trace_id: str
```

```
    tenant_id: str
```

```
    domain: str
```

```
    # Governance
```

```
    policy_id: str
```

```
    policy_version: int
```

```
    model_version: str
```

```
    prompt_version: str
```

```
    guardrail_version: str
```

```
    verifier_version: str
```

```
    complexity_scorer_version: str
```

```
    # Decision
```

```
    complexity_score: float
```

```
    fast_path_used: bool
```

```
    base_budget_class: str
```

```
    effective_budget_class: str
```

```
    downgrade_reason: Optional[str]
```

```
    priority_tier: str
```

```
    max_reasoning_tokens: int
```

```
    route: str
```

```
    cache_hit: bool
```

```
    admission_decision: str
```

```
    # Execution
```

```
    reasoning_tokens_used: int
```

```
    output_tokens: int
```

```
    tokens_saved: int
```

```
    stop_reason: str      # natural_boundary | budget_exhausted |
```

cache_hit | fast_path_default

escalation_count: int

fallback_used: bool

Guardrails

guardrail_input_result: str

guardrail_input_ms: int

guardrail_injection_score: float

guardrail_output_result: str

guardrail_output_ms: int

Verification

verification_result: str

verification_type: str

verification_ms: int

Latency breakdown

queue_ms: int

prefill_ms: int

decode_ms: int

ttft_ms: int

e2e_latency_ms: int

Cost

estimated_cost_usd: float

actual_cost_usd: float

Circuit breakers

gpu_cb_state: str

latency_cb_state: str

Entropy telemetry (sampled)

entropy_samples: list[float]

Prometheus Metric Catalog

All custom metrics use the ic_ prefix (InferaCordon). vLLM native metrics are consumed directly without renaming.

Cost metrics

ic_request_cost_usd{tenant, budget_class, route} histogram

ic_tokens_reasoning_used{tenant, budget_class} histogram

ic_tokens_reasoning_saved{tenant, budget_class} histogram

ic_cost_per_correct_answer{tenant, task_class} gauge (updated by eval worker)

ic_cache_hits_total{tenant} counter

ic_cache_misses_total{tenant} counter

Quality metrics

ic_verification_result{tenant, verifier_type, result} counter
ic_escalation_count{tenant, from_class, to_class} counter
ic_guardrail_block_total{tenant, stage, reason} counter
ic_judge_score{tenant, task_class} histogram (from eval worker)

Reliability metrics

ic_circuit_breaker_state{breaker_name} gauge (0=closed, 1=open)
ic_fallback_total{tenant, reason} counter
ic_admission_rejected_total{tenant, reason} counter
ic_fast_path_total{tenant} counter
ic_request_latency_ms{tenant, budget_class, route} histogram

Governance metrics

ic_policy_version{tenant, domain, version} gauge
ic_rate_limit_rejected_total{tenant, limit_type} counter
ic_budget_class_distribution{tenant, class} counter
ic_audit_write_dropped_total counter

vLLM native (consumed directly)

vllm:gpu_cache_usage_perc gauge
vllm:num_requests_running gauge
vllm:num_requests_waiting gauge
vllm:time_to_first_token_seconds histogram
vllm:time_per_output_token_seconds histogram

Grafana Dashboard Structure

One dashboard JSON file: dashboards/grafana/inferacordon.json. Pre-configured with all panels. Loaded automatically at Grafana container startup via provisioning. Zero manual configuration required.

Five tabs:

Cost: Bar chart of cost per request by budget class (last hour). Large metric card showing cost per correct answer with delta vs. natural completion baseline. Donut chart of cache hit rate with hit and miss counts. Bar chart of tokens saved per request by budget class. Time series of total hourly spend by tenant.

Quality: Stacked bar chart of verification pass/fail/escalation rates by task class. Line chart of rolling LLM judge score average with policy quality floor as dashed reference line. Table of last 20 verification results with request ID, task class, budget class, result, and trace link.

Reliability: Three circuit breaker state indicators (GPU Pressure, Latency, Admission) with current metric values and time since last state change. Line chart of P95 TTFT over last 30 minutes. Gauge of current queue depth vs. admission ceiling. Table of last 10 circuit breaker transition events.

Performance: GPU utilization gauge (live, 5s refresh). KV-cache utilization gauge (live, 5s refresh). Line chart of decode throughput in tokens/second. TTFT histogram with P50, P95, P99 markers. Fast-path activation rate counter.

Governance: Table of active policy version per tenant with last-changed timestamp. Chronological event log of governance events (last hour): budget class downgrades, guardrail blocks, fast-path

activations, circuit breaker transitions, rate limit hits, audit write drops. Bar chart of budget class distribution for today's traffic.

Component 9: Frontend — InfaCordon Console

Build and Serving

The React application is built during the Docker image build step using `npm run build`. The output `dist/` directory is copied into the gateway container and served as static files from the FastAPI route `GET /`. The frontend calls the gateway API on the same origin — no CORS configuration, no separate frontend server, no additional container.

Design Tokens

```
:root {
  --color-bg-base:    #0D1117; /* page background */
  --color-bg-surface: #161B22; /* cards, panels */
  --color-bg-elevated: #1C2128; /* modals, dropdowns */
  --color-border:     #21262D; /* card borders */
  --color-border-focus: #388BFD; /* focused inputs */

  --color-accent-primary: #6366F1; /* indigo — primary CTA */
  --color-accent-secondary: #818CF8; /* lighter indigo — hover states */
  --color-accent-glow:    rgba(99, 102, 241, 0.15); /* focus glow */

  --color-text-primary: #F0F6FC;
  --color-text-secondary: #8B949E;
  --color-text-disabled: #484F58;

  --color-status-healthy: #10B981; /* emerald */
  --color-status-warning: #F59E0B; /* amber */
  --color-status-error:   #EF4444; /* rose */
  --color-status-info:    #38BDF8; /* sky blue */

  --font-ui: 'Inter', system-ui, sans-serif;
  --font-mono: 'JetBrains Mono', 'Fira Code', monospace;

  --radius-card: 8px;
  --radius-button: 6px;
  --radius-input: 6px;

  --shadow-card: 0 1px 3px rgba(0,0,0,0.4);
  --shadow-focus: 0 4px 16px var(--color-accent-glow);

  --transition-fast: 100ms ease-out;
  --transition-normal: 150ms ease-out;
}
```

Page Specifications

Page 1: Inference Playground

The demo centerpiece. The page is split: left panel 60%, right panel 40%, with a 1px divider at `--color-border`.

Left panel top section: A labeled textarea with placeholder "Enter your prompt..." The textarea has a bottom-right token count estimate displayed in `--font-mono` at `--color-text-secondary`. Below the textarea: a horizontal row containing a tenant selector (dropdown, styled select component), a domain selector, and an estimated cost ceiling pulled from the policy for the selected tenant displayed as a pill badge.

Below the selectors: a settings row with two toggles. "Let platform decide" toggle (on by default) controls whether the budget class override selector appears. A "Require verification" toggle controls whether the request body includes the verification flag. Both toggles are styled with the standard InfaCordon toggle component: dark track at `--color-bg-elevated`, emerald thumb when active. On submit: the textarea and settings row slide up and a stage progress tracker slides in. The tracker shows five stages as a horizontal pipeline: Complexity Scoring, Policy + Guardrail, Inference, Verification, Response. Each stage shows its label and, when complete, its measured latency in milliseconds in `--font-mono` at `--color-accent-secondary`. The active stage shows a 2px animated progress bar at `--color-accent-primary`. A failed stage shows in `--color-status-error` with an error label. Response area: After the inference stage completes, the response text renders in a markdown-formatted content area using a lightweight markdown renderer. Below the response, the metrics strip: seven chips in a horizontal row. Each chip has a label in `--font-ui` at `--color-text-secondary` (10px, uppercase, letter-spaced) and a value in `--font-mono` at `--color-text-primary`. The budget class chip is a colored pill: emerald for low, indigo for medium, amber for high, rose for critical. The stop reason chip is text with a small icon. The verification result chip shows a checkmark or X in the appropriate status color.

Right panel: Live trace waterfall. Spans appear as horizontal bars as each stage completes. The timeline starts at $t=0$ (request receipt) and extends rightward. Each span bar shows its label inside (if wide enough) or as a tooltip on hover. Span colors: gateway stages in `--color-accent-primary` at 60% opacity, policy engine in `--color-text-secondary` at 60% opacity, guardrail stages in `--color-status-healthy` for pass and `--color-status-warning` for flag, inference in `--color-accent-secondary`, verifier in `--color-status-info`, escalation retry in `--color-status-warning`, circuit breaker event in `--color-status-error`. An escalation shows as a gap and a second set of inference and verifier spans, clearly communicating the retry.

Below the waterfall: a "Raw Trace JSON" collapsible section with a syntax-highlighted JSON viewer showing the complete trace record.

Page 2: Observability Dashboard

Five tab panels. Data pulled from the Prometheus HTTP API (`/api/v1/query` and `/api/v1/query_range`) and the gateway's `/v1/eval-results` endpoint. All charts rendered with Recharts.

Each tab panel has a header row with a title, a time range selector (last 1h, 6h, 24h), and a refresh indicator showing the last data fetch timestamp.

Chart components use consistent styling: axis labels in `--font-mono` at 11px `--color-text-secondary`, chart lines and bars in the accent and status palette, grid lines at `--color-border` at 40% opacity, tooltips on a `--color-bg-elevated` background with `--color-border` border.

Page 3: Policy Manager

A two-column layout. Left column: a list of loaded policies as cards, each showing the policy ID, tenant, domain, and version number. Clicking a policy loads it in the right column.

Right column: The selected policy rendered as a structured card with four collapsible sections: Budget Profiles (a compact table with columns Class, Model, Max Reasoning Tokens, Max Output Tokens, Verification), Guardrail Configuration (key-value pairs), Limits (key-value pairs), and Version Identifiers (all version strings in `--font-mono`).

Below the policy card: a diff viewer showing changes between the current version and the previous version if both are available. Additions in emerald on a dark emerald background. Removals in rose on a dark rose background. Unchanged lines in `--color-text-secondary`.

At the bottom: a YAML validation panel with a code editor (using a lightweight syntax-highlighting textarea). As the user types, the frontend calls `POST /admin/validate-policy` and displays either a success message with the parsed policy summary or a structured error list with field paths, expected types, and violation descriptions. The error list uses the standard IntraCordon alert component: rose border, dark rose background, error icon, error message in `--font-mono`.

Page 4: Audit Log Explorer

Full-width table with horizontal scroll for overflow. Columns: Timestamp (`--font-mono`), Tenant, Request ID (`--font-mono`, truncated with hover tooltip), Budget Class (colored pill), Route, Stop Reason, Reasoning Tokens (used/allocated in `--font-mono`), Verification (icon), Escalation Count, Cost USD (`--font-mono`).

Filter bar above the table: tenant dropdown, budget class multi-select, stop reason multi-select, verification result toggle group, date range picker, and a text search field for request ID prefix search. All filters apply client-side against the loaded page of records, with server-side pagination triggered when the user scrolls to the table bottom.

Selecting a row expands an inline detail panel below the row (not a modal — inline, preserving table context). The detail panel shows three sections: Decision Record (full JSON in syntax-highlighted viewer), Outcome Record (full JSON), and Linked Resources (trace ID linking to Jaeger, evaluation result if available, policy version linking to the Policy Manager view).

Page 5: System Health

Six component status cards in a 3×2 grid: Gateway, vLLM Serving Layer, Guardrail Service, Verifier Service, Evaluation Worker, Semantic Cache. Each card shows: a large status indicator dot (emerald for operational, amber for degraded, rose for down), the component name, current request rate (requests/minute), error rate (%), P95 latency, and a small sparkline of P95 latency over the last 10 minutes.

Below the component cards: two circuit breaker status panels side by side. GPU Pressure Circuit Breaker and Latency Circuit Breaker. Each panel shows: state label (CLOSED in emerald or OPEN in rose), the current metric value, the threshold, and the time since last state transition. A mini history chart shows the last 20 state changes as a step function.

At the bottom of the page: the Failure Injection Panel. Visible only when the environment variable `INFERACORDON_DEMO_MODE=true` is set. Ten buttons arranged in a 5×2 grid, each labeled with the failure mode name. Clicking a button sends `POST /admin/inject-failure` with the failure type and streams the result back. A result panel below the buttons shows: which failure was injected, what system response was observed, what the client received, and what was written to the audit log. Each result is shown as a card with a status indicator and a collapsible detail section showing the relevant audit record.

This panel is the most compelling demo element. Injecting a GPU pressure event, watching the circuit breaker indicator flip to OPEN on the Health page, seeing the next request's trace show a budget class downgrade, and then watching the breaker close again as the injected pressure is removed — all in

real time — demonstrates the reliability engineering of the system more powerfully than any static documentation.

Failure Mode Catalog (Implementation Reference)

Each failure mode has four attributes: the detection mechanism, the handler location, the system response, and the observable effect in the frontend. All ten are tested before the demo is recorded.

Failure 1: Complexity scorer exceeds 5ms Detection: Deadline timer in pipeline orchestrator.

Handler: pipeline/orchestrator.py Response: Set fast_path_used=True, assign medium budget class, set complexity_score=-1, continue. Observable: Yellow "Fast Path" badge on Playground trace. ic_fast_path_total counter incremented.

Failure 2: LogitProcessor raises exception Detection: Exception handler in LogitProcessor __call__.

Handler: vllm_adapter/logit_processor.py Response: Log exception, disable adaptive stopping for this request, run to hard max_tokens limit, record stop_reason=budget_exhausted_fallback. Observable: Amber span in trace waterfall labeled "Processor Error — Hard Cap Applied".

Failure 3: GPU KV-cache utilization exceeds 90% Detection: Circuit breaker polling vLLM

Prometheus every 5 seconds. Handler: vllm_adapter/circuit_breakers.py Response: Open GPU pressure circuit breaker. Route all new low-priority requests to cheap model. Observable: Rose "OPEN" state on GPU Pressure circuit breaker card on Health page. Budget class decision records show route=cheap_model_cb_redirect.

Failure 4: P99 TTFT exceeds SLO threshold Detection: Circuit breaker evaluating vLLM TTFT

histogram every 30 seconds. Handler: vllm_adapter/circuit_breakers.py Response: Open latency circuit breaker. Assign all new requests one budget class lower than calculated. Observable: Amber "OPEN" state on Latency circuit breaker card. ic_circuit_breaker_state{breaker_name="latency"} gauge = 1.

Failure 5: Guardrail service HTTP call exceeds 200ms Detection: HTTP client timeout in

clients/guardrail_client.py. Handler: clients/guardrail_client.py Response: Return 503 to client with error: guardrail_timeout. Write audit event. Increment ic_guardrail_block_total{reason="timeout"}. Observable: Rose response card in Playground. Audit Log shows guardrail_timeout event. Health page shows guardrail service error rate spike.

Failure 6: Output safety classifier returns unsafe Detection: Guardrail service safety classifier

result. Handler: gateway/main.py after inference, before response delivery. Response: Return structured refusal body with confidence_level: blocked and audit_reference: <request_id>. Write audit event. Observable: Rose response card in Playground with audit reference ID. Audit Log shows safety_block event for output stage.

Failure 7: Policy lookup returns no matching policy Detection: KeyError in policy engine lookup.

Handler: policy_engine/loader.py Response: Load default policy default_v1. Set policy_id=default in all records. Log audit event with reason=no_policy_match. Observable: Amber "Default Policy" badge in Playground trace. Governance dashboard shows default policy usage counter increment.

Failure 8: Rate limit exhausted Detection: Token bucket empty in rate limiter. Handler:

gateway/middleware/rate_limiter.py Response: Return 429 with Retry-After header and structured body specifying which limit and reset time. Observable: Playground shows error state with countdown timer to retry window. ic_rate_limit_rejected_total counter incremented.

Failure 9: OpenTelemetry export fails Detection: Exception in OTel exporter. Handler:

telemetry/otel_config.py exporter exception handler. Response: Buffer trace to in-memory ring buffer

(capacity 500 traces). Retry export on next flush interval. Never block the serving path. Observable: Warning indicator on Health page showing buffered trace count. No client-visible impact.

Failure 10: Cheap model (fallback) unavailable Detection: HTTP client error or timeout from vLLM on cheap model request. Handler: clients/vllm_client.py Response: Return 503 with error: fallback_unavailable and retry_after_seconds. Write audit event. Observable: Error state in Playground. Audit Log shows fallback_unavailable event. Health page shows vLLM error rate spike.

Evaluation Framework (Implementation Reference)

Benchmark Harness Structure

evaluation/
└─ benchmark_harness.py ← Main runner; orchestrates all datasets
└─ datasets/
 └─ gsm8k_loader.py
 └─ math500_loader.py
 └─ humaneval_loader.py
 └─ enterprise_synthetic.jsonl ← 200 manually written examples
 └─ adversarial.jsonl ← 50 failure-mode trigger examples
└─ baselines.py ← Runs all 6 baselines
└─ ablation_runner.py ← Runs each system layer incrementally
└─ regression_gate.py ← CI gate; fails on SLO violation
└─ llm_judge.py ← Async judge; GPT-4o-mini
└─ cost_reporter.py ← Computes C_online and C_total
└─ update_readme_metrics.py ← Patches README with measured values

Six Baselines (Evaluated in Order)

The baselines are evaluated before any optimization component is active. Each baseline is a configuration of the benchmark harness that bypasses components selectively.

Baseline 1 — Natural completion: LogitProcessor disabled. No budget ceiling beyond vLLM's server maximum. All requests on reasoning model.

Baseline 2 — Fixed low budget: Hard max_tokens on reasoning set to 25th percentile of Baseline 1 token distribution.

Baseline 3 — Fixed medium budget: Hard max_tokens set to 50th percentile.

Baseline 4 — Policy routing only: Budget class selected by policy and complexity scorer. No LogitProcessor. No escalation.

Baseline 5 — Policy routing with escalation: Same as Baseline 4 plus verification-triggered escalation.

Baseline 6 — Full IntraCordon: All components active including LogitProcessor, escalation, circuit breakers, and semantic cache.

Ablation Study Table

The ablation shows the marginal contribution of each component to the primary metric.

System Configuration C_online Accuracy P95 Latency

Baseline 1: Natural completion	[measured]	[measured]	[measured]
Baseline 2: Fixed low budget	[Δ]	[Δ]	[Δ]
Baseline 3: Fixed medium budget	[Δ]	[Δ]	[Δ]
+ Complexity scorer (policy routing)	[Δ]	[Δ]	[Δ]

+ LogitProcessor stopping	[Δ]	[Δ]	[Δ]
+ Verification escalation	[Δ]	[Δ]	[Δ]
+ Semantic cache	[Δ]	[Δ]	[Δ]
Full InteraCordon	[final]	[final]	[final]

Each Δ is the marginal change from the row above. Components that do not improve C_online are documented honestly.

Regression Gate

The gate runs as a CI step triggered by any configuration change. It evaluates the full benchmark suite against the current configuration and compares per-slice accuracy against the stored baseline.

```
def check_regression(
    current_results: BenchmarkResults,
    baseline_results: BenchmarkResults,
    policy: PolicyDocument
) -> GateResult:

    for task_class in current_results.task_classes:
        delta = (
            current_results.accuracy[task_class]
            - baseline_results.accuracy[task_class]
        )
        threshold = (
            -policy.slo.quality_regression_pp / 100.0
        )
        if delta < threshold:
            return GateResult(
                passed=False,
                failed_class=task_class,
                delta=delta,
                threshold=threshold
            )

    return GateResult(passed=True)
```

The gate evaluates per-slice, not aggregate. A configuration that improves average accuracy while degrading one task class fails the gate.

Implementation Roadmap (Fixed)

Week 1-2: Foundation and Validation Gate

Week 1, Days 1-3: Docker Compose setup with all containers. vLLM running both models. Health checks passing. Prometheus scraping vLLM native metrics. Grafana loading (empty dashboard).

Week 1, Days 4-5: LogitProcessor implementation. End-of-thinking token ID identified for DeepSeek-R1-7B. Validation test run: 50 diverse queries, delimiter injection at five token positions, coherence scored by LLM judge. Gate: 85%+ coherence rate. If gate fails: implement hard max_tokens fallback and document the switch.

Week 2: Run all six baselines. Collect natural completion token distributions. Calibrate complexity scorer weights using linear regression against observed token usage. Record cost per correct answer for each baseline. This is the ground truth. Implement skeleton FastAPI gateway with auth, rate limiter, trace injection, and Prometheus metrics endpoint.

Milestone: Working inference service, validated stopping mechanism, measured baselines, calibrated complexity scorer.

Weeks 3-4: Policy Engine and Budget Control

Build the YAML policy engine with Pydantic validation and startup-time validation enforcement. Implement the budget controller and priority tier assignment. Implement the audit log writer with the decision and outcome record schemas. Implement the semantic cache with FAISS. Implement static prompt rewriting. Implement the fast-path circuit breaker. Run the evaluation with policy routing active (Baseline 4) and measure the marginal improvement.

Milestone: Every request receives a governed budget and a complete audit record. The fast-path fires correctly under induced delay.

Weeks 5-6: Guardrails and Verifier

Build the guardrail service container. Export injection classifier and safety classifier to ONNX. Validate both models against test inputs before touching production traffic. Implement Presidio PII redaction in the gateway. Implement output guardrail check. Build the verifier service container wrapping official evaluation harnesses. Validate each verifier against 10 known-correct and 10 known-incorrect examples. Implement verification-triggered escalation. Run Baseline 5 evaluation and measure marginal improvement.

Milestone: All four pillars have at least one working implementation. Guardrails: input and output safety. Governance: policy engine and audit log. Cost: budget routing and escalation. Inference: LogitProcessor stopping.

Weeks 7-8: Reliability Layer

Implement GPU pressure and latency circuit breakers with Prometheus polling. Implement admission control semaphore. Implement all ten failure mode handlers. Write one test per failure mode in tests/failure_injection/. Run each test and verify the documented behavior. Fix any failures. Implement local telemetry buffering for trace backend unavailability. Run load test at 10 RPS and 50 RPS and record the capacity profile.

Milestone: Every failure mode produces documented behavior. Load test shows stable operation at target RPS.

Weeks 9-10: Complete Observability

Complete the OpenTelemetry trace schema and validate that every field is populated for a sample request. Configure Jaeger. Implement the async evaluation worker thread pool. Implement LLM-as-judge scoring and SQLite writes. Build and configure the complete Grafana dashboard from the committed JSON file. Implement update_readme_metrics.py. Run the full benchmark suite (all six baselines plus full InferenceCordon) and generate the complete cost-quality-latency report.

Milestone: All metrics produced by benchmark scripts. README updated with measured values.

Weeks 11-12: Frontend

Build all five React pages with the complete design language applied. Wire all API calls and Prometheus queries. Validate the Playground with all three representative request paths (simple query, complex query, escalation). Validate the Failure Injection Panel for all ten failure modes. Validate the Audit Log Explorer with filters and row expansion. Validate the Policy Manager validation panel. Run the five-minute demo scenario end-to-end and record timing.

Milestone: Frontend is demo-ready. All pages functional. Design language applied consistently.

Weeks 13-14: Packaging and Portfolio

Write the technical design document. Record the five-minute demo video. Finalize the architecture diagram. Write the portfolio case study (problem, approach, results, lessons learned). Run `update_readme_metrics.py` one final time with production benchmark numbers. Review and polish all documentation. Tag the release.

Milestone: Complete, documented, demo-ready system with measured results.

Repository Structure (Final)

```
inferacordon/
|
|— docker-compose.yml
|— .env.example
|— README.md           ← Auto-updated
|— architecture.md      ← This document (concise version)
|
|— gateway/
|   |— Dockerfile
|   |— requirements.txt
|   |— main.py
|   |— middleware/
|   |   |— auth.py
|   |   |— rate_limiter.py
|   |   └─ trace_injector.py
|   |— pipeline/
|   |   |— orchestrator.py
|   |   |— complexity_scorer.py
|   |   |— pii_redactor.py
|   |   └─ context_engine.py
|   |— routing/
|   |   |— budget_controller.py
|   |   |— model_router.py
|   |   |— admission_control.py
|   |   └─ semantic_cache.py
|   |— clients/
|   |   |— vllm_client.py
|   |   |— guardrail_client.py
|   |   └─ verifier_client.py
|   |— escalation/
|   |   └─ escalation_handler.py
|   └─ evaluation/
|       └─ eval_worker.py
|
|— policy_engine/
```

```
|  ├── loader.py
|  ├── validator.py
|  ├── budget_controller.py
|  ├── audit_log.py
|  └── policies/
|      ├── acme_corp_general_qa_v1.yaml
|      └── demo_tenant_v1.yaml
|
|  ├── vllm_adapter/
|  |   ├── logit_processor.py
|  |   ├── circuit_breakers.py
|  |   ├── constants.py          ← End-of-thinking token IDs
|  |   └── serving_config.py
|  |
|  ├── guardrail_service/
|  |   ├── Dockerfile
|  |   ├── requirements.txt
|  |   ├── main.py
|  |   ├── injection_classifier.py
|  |   ├── safety_classifier.py
|  |   └── models/              ← Docker volume mount point
|  |
|  ├── verifier_service/
|  |   ├── Dockerfile
|  |   ├── requirements.txt
|  |   ├── main.py
|  |   ├── gsm8k_verifier.py
|  |   ├── math_verifier.py
|  |   ├── humaneval_runner.py
|  |   └── schema_verifier.py
|  |
|  ├── telemetry/
|  |   ├── otel_config.py
|  |   ├── prometheus_metrics.py
|  |   └── trace_schema.py
|  |
|  ├── evaluation/
|  |   ├── benchmark_harness.py
|  |   ├── baselines.py
|  |   ├── ablation_runner.py
|  |   ├── regression_gate.py
|  |   ├── llm_judge.py
|  |   └── cost_reporter.py
```

```
| |— update_readme_metrics.py
| |— datasets/
| |   |— gsm8k_loader.py
| |   |— math500_loader.py
| |   |— humaneval_loader.py
| |   |— enterprise_synthetic.jsonl
| |   |— adversarial.jsonl
|
| |— load_testing/
| |   |— locustfile.py
| |   |— workload_profiles.py
|
| |— tests/
| |   |— unit/
| |   |— integration/
| |   |— failure_injection/
| |   |   |— test_fast_path.py
| |   |   |— test_logit_processor_exception.py
| |   |   |— test_gpu_pressure_cb.py
| |   |   |— test_latency_cb.py
| |   |   |— test_guardrail_timeout.py
| |   |   |— test_output_safety_block.py
| |   |   |— test_policy_miss.py
| |   |   |— test_rate_limit.py
| |   |   |— test_otel_export_failure.py
| |   |   |— test_fallback_unavailable.py
|
| |— frontend/
| |   |— package.json
| |   |— vite.config.js
| |   |— src/
| |   |   |— App.jsx
| |   |   |— design/
| |   |   |   |— tokens.css
| |   |   |— pages/
| |   |   |   |— Playground.jsx
| |   |   |   |— Dashboard.jsx
| |   |   |   |— PolicyManager.jsx
| |   |   |   |— AuditLog.jsx
| |   |   |   |— SystemHealth.jsx
| |   |   |— components/
| |   |   |   |— TraceWaterfall.jsx
| |   |   |   |— MetricsStrip.jsx
```


Reliability engineering — The failure mode catalog with tested, documented, and observable failure behaviors demonstrates SRE-grade thinking. The circuit breakers, admission control, and local telemetry buffering demonstrate that reliability is designed in, not bolted on.

Governance and compliance — The versioned, immutable policy system with per-request audit records referencing exact component versions demonstrates enterprise-grade governance. The Audit Log Explorer makes this legible to non-technical stakeholders.

Cost engineering — Cost per correct answer as a primary metric, the semantic cache, the budget class routing, and the ablation study showing the marginal cost contribution of each component demonstrate rigorous cost engineering.

Observability — The complete OpenTelemetry trace schema, the Prometheus metric catalog, the pre-configured Grafana dashboard, and the frontend's live dashboard pulling from Prometheus directly demonstrate production-grade observability.

Evaluation rigor — The six-baseline comparison, the per-slice regression gate, the ablation study, and the auto-updating README demonstrate that claims are measured, not asserted.